

# Projet de Programmation Orientée Objet

## Implémentation du jeu *Blue Prince* en Python

Parcours : Syscom

Binôme : Khadim GAYE & Cheikh Omar Ndao

Université : Sorbonne Université

Année / 2025 – 2026

## Sommaire :

**1/Introduction**

**2/Architecture générale et choix de conception**

**3/Fonctionnalités implémentées**

**4/Justification des choix (héritage, abstraction, modularité)**

**5/ Interface graphique**

**6/ Gestion de l'aléatoire**

**7/Effets spéciaux et interactions**

**8/ Gestion de projet**

**9/ Conclusion**

# I/Introduction

Dans le cadre du module de Programmation Orientée Objet (POO), il nous a été demandé de réaliser une version simplifiée du jeu Blue Prince en Python. Ce projet constitue une mise en pratique des concepts fondamentaux de la programmation orientée objet étudiés en cours, tels que la structuration en classes, l'héritage, l'abstraction et la modularité.

L'objectif principal est de concevoir un programme fonctionnel et documenté, intégrant une interface graphique avec pygame, permettant au joueur de progresser dans un manoir généré dynamiquement. Le projet vise à développer plusieurs compétences :

- Techniques : maîtrise de la POO en Python, conception d'algorithmes, gestion de l'aléatoire et des interactions entre objets.
- Pratiques : organisation du code, documentation claire, utilisation de GitHub pour le travail collaboratif.
- Pédagogiques : justification des choix de conception et capacité à présenter et défendre l'implémentation lors de l'examen oral.

Ce rapport a pour but de présenter notre démarche de conception, d'expliquer les choix retenus pour la modélisation du jeu, et de fournir un diagramme UML exhaustif décrivant la structure du programme. Il servira également de support pour démontrer la cohérence entre notre code, les fonctionnalités demandées et les critères d'évaluation définis dans le cahier des charges.

## II/ Architecture générale et choix de conception

Dans ce projet, nous avons adopté une architecture modulaire inspirée du modèle MVC (Modèle-Vue-Contrôleur) afin de garantir une séparation claire des responsabilités. Ce choix permet de structurer le programme en trois grandes composantes : le modèle, qui regroupe les données et la logique métier du jeu (classes Manoir, Inventaire, Piece) ; la vue, assurée par l'interface graphique via pygame, qui affiche la grille du manoir, les pièces et l'inventaire ; et enfin le contrôleur, incarné par la classe Controleur, qui gère les interactions clavier et orchestre les actions du joueur. Ce découpage facilite la maintenabilité, l'extensibilité et la testabilité du code.

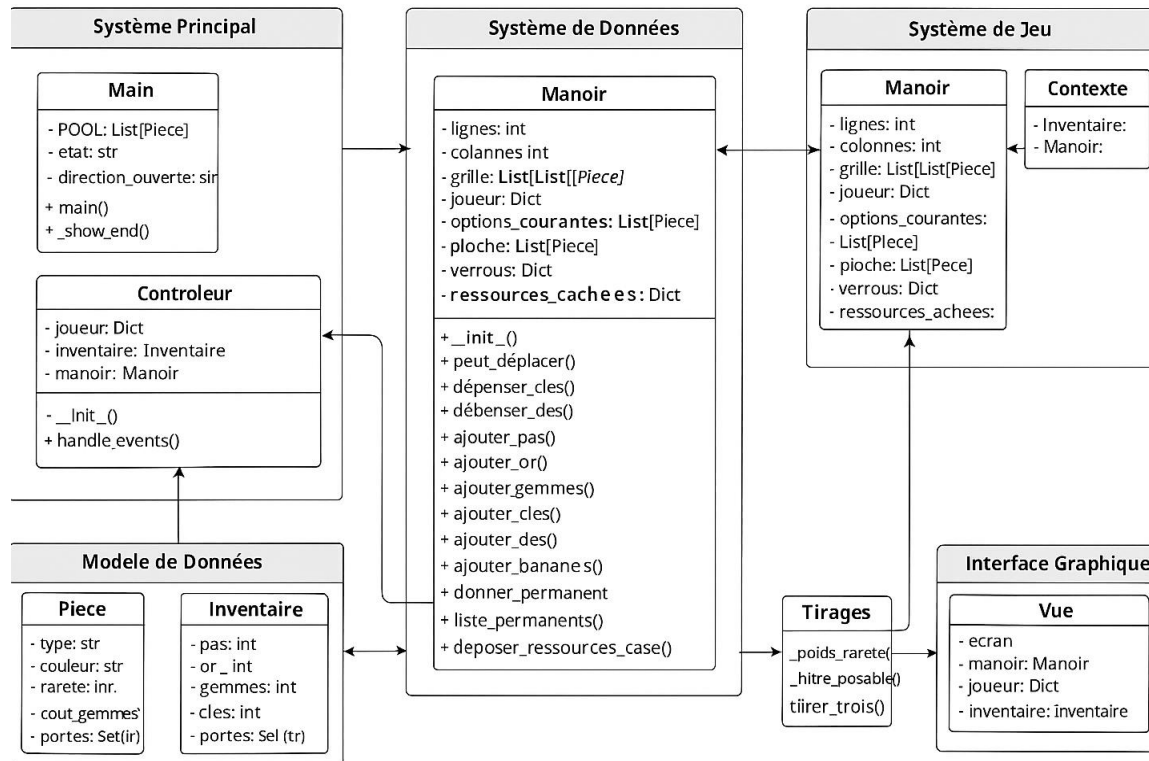
Le diagramme UML est structuré en trois sous-systèmes :

- **Système Principal** : classes Main (point d'entrée, initialisation, états globaux) et Controleur (gestion des événements, commandes clavier, déplacements, tirages).
- **Système de Données** : classes Manoir (grille de jeu, logique centrale, conditions de victoire/défaite), Inventaire (ressources consommables et objets permanents), et Piece (caractéristiques des salles : type, couleur, rareté, coût, effets).

- **Système de Jeu** : classes Tirages (tirage aléatoire des pièces avec contraintes et rareté) et Contexte (état du manoir, inventaire, position du joueur, transmission aux fonctions de gameplay).

Ce découpage clarifie les responsabilités, limite les dépendances et assure la cohérence globale du programme.

## Le DIAGRAMME UML :



## III/ Fonctionnalités implémentées

Nous avons implémenté tous les effets spéciaux demandés dans l'énoncé :

- effets lors de l'entrée dans une salle,
- effets lors du tirage d'une salle,
- modification de probabilités de tirage de pièces,
- modification de probabilités de loot,

- dispersion d’objets dans d’autres salles,
- ajout de nouvelles pièces au catalogue.

De plus, nous avons ajouté un effet spécial visuel original lors de la fin de partie : extinction progressive du manoir, chute du joueur, puis apparition en fondu du message final.

Cette implémentation complète nous permet d'offrir une expérience de jeu riche et stratégique, fidèle à l'esprit du Blue Prince original tout en respectant scrupuleusement les exigences du projet.

## **IV. Justification des choix (héritage, abstraction, modularité)**

### **Héritage**

Nous avons utilisé l’héritage pour factoriser les comportements communs entre certaines classes. Par exemple, les différentes **pièces** du manoir partagent des attributs (type, couleur, rareté, coût en gemmes, effets), ce qui justifie une classe mère Piece dont héritent les sous-classes spécialisées. Cela permet de réutiliser du code, d’éviter les duplications et de faciliter l’ajout de nouvelles pièces avec des comportements spécifiques.

### **Abstraction**

L’abstraction a été mise en œuvre pour séparer les **concepts généraux** des détails d’implémentation. Les classes comme Controleur ou Tirages définissent des méthodes abstraites de gestion (tirer une pièce, interpréter une commande) sans exposer directement la logique interne. Cette approche rend le code plus lisible et permet de modifier ou d’étendre certaines fonctionnalités sans impacter l’ensemble du système.

### **Modularité**

La modularité est assurée par une organisation en **sous-systèmes indépendants** (Système Principal, Système de Données, Système de Jeu). Chaque module regroupe des responsabilités précises :

- **Main / Controleur** : lancement et gestion des événements.
- **Manoir / Inventaire / Piece** : logique du jeu et stockage des ressources.
- **Tirages / Contexte** : gestion de l’aléatoire et transmission des états.

Cette séparation facilite la maintenance, les tests unitaires et l’évolution du projet. Elle permet aussi un travail collaboratif efficace, chaque membre pouvant se concentrer sur un module sans perturber les autres.

## **V/ Interface graphique**

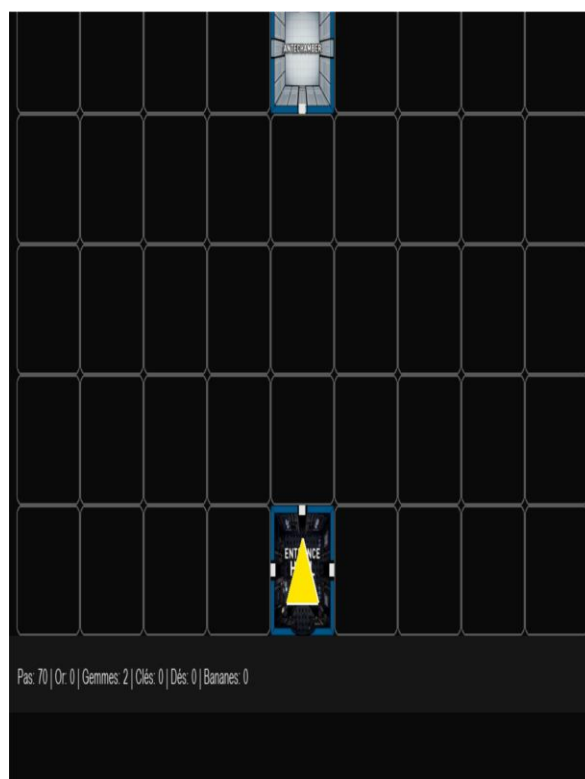
L'interface graphique a été développée avec pygame, qui permet l'affichage des éléments visuels et la gestion des interactions clavier. La grille du manoir est représentée par une matrice de cases : les pièces découvertes apparaissent avec leur image, les cases inexplorées restent noires, et le joueur est indiqué par une flèche orientée selon sa direction. Les déplacements se font via les touches ZQSD, tandis que l'ouverture des portes et la sélection des pièces utilisent "espace" et "entrée".

Un bandeau HUD en bas de l'écran affiche en temps réel les ressources du joueur (pas, or, gemmes, clés, dés) ainsi que les objets permanents. Lorsqu'une porte est ouverte, un menu propose trois pièces tirées au sort, présentées avec leur image, leur nom et leur coût en gemmes.

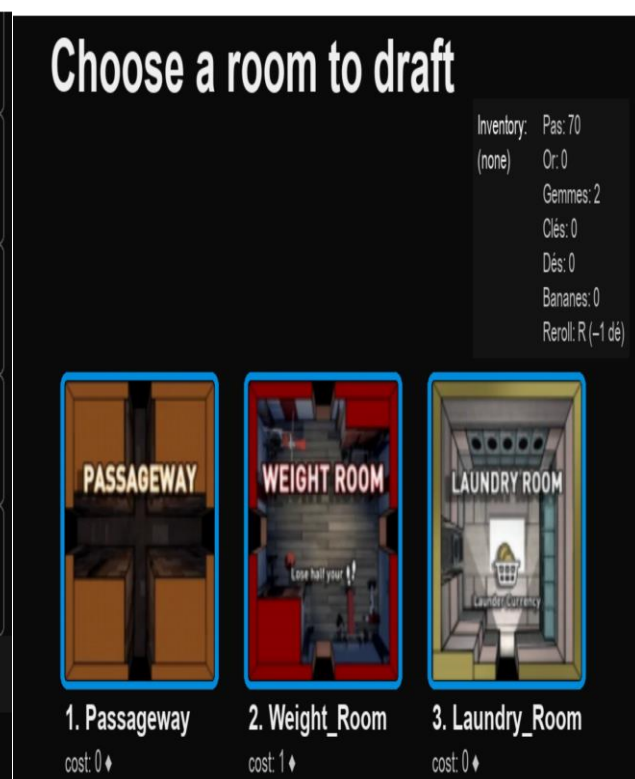
L'interface a été conçue pour rester claire et ergonomique : couleurs cohérentes avec le type de pièce, textes lisibles et effets visuels simples. Elle est également extensible, permettant d'ajouter facilement de nouvelles pièces ou objets.

Les images obtenues :

1-Au départ



2-Le choix des pièces



### 3- les avancements



### 4-Si vous avez gagné



### 5 - Si vous avez perdu



## VI/ Gestion de l'aléatoire

La gestion de l'aléatoire est au cœur du projet et assure une rejouabilité élevée. Le module **random** de Python est utilisé pour :

- Portes : leur statut (niveau 0, 1 ou 2) est tiré aléatoirement, avec une difficulté croissante au fur et à mesure de la progression dans le manoir.
- Pièces : le tirage est pondéré par la rareté (0 à 3), garantissant au moins une option gratuite si le joueur n'a pas de gemmes. Les dés permettent de relancer le tirage.
- Objets : coffres, casiers et endroits à creuser génèrent des ressources variées de manière aléatoire. Les objets permanents et certains effets de pièces modifient ces probabilités.
- Effets spéciaux : certaines pièces influencent directement les tirages (ex. Greenhouse pour les pièces vertes, Furnace pour les rouges).

Ainsi, l'aléatoire n'est pas un simple hasard : il équilibre la difficulté, évite les blocages et rend chaque partie unique.

## VII/ Effets spéciaux et interactions

Les effets spéciaux enrichissent le gameplay en modifiant les ressources, les probabilités ou la structure du manoir :

- Ressources : certaines pièces ajoutent ou retirent des pas, des clés, des dés ou de l'or (ex. Serre, Fournaise, Weight Room).
- Magasins : les pièces jaunes permettent d'échanger de l'or contre des objets ou des pas.
- Distribution : certaines salles dispersent des ressources dans le manoir (ex. Patio pour l'or, Office pour les gemmes).
- Probabilités : des pièces influencent les tirages (ex. Greenhouse augmente les pièces vertes, Veranda accroît les chances de trouver des gemmes).

Ces effets et interactions rendent chaque partie unique, obligent le joueur à adapter sa stratégie et renforcent la rejouabilité du jeu.

## VIII/ Gestion de projet

Le projet a été mené de façon collaborative grâce à un dépôt GitHub public, utilisé pour centraliser le code et assurer un suivi régulier des contributions. Chaque membre du binôme a participé activement par des commits fréquents et bien nommés, en respectant une organisation incrémentale : d'abord les fonctionnalités de base (déplacement, inventaire, tirage), puis les fonctionnalités avancées (effets spéciaux, gestion de l'aléatoire, objets permanents).

### Répartition des tâches

- Binôme Cheikh : responsable du développement du manoir ([manoir.py](#)), du système de tirage des pièces ([tirage.py](#)), de la gestion du catalogue ([catalogue.py](#)) et Requirement.
- Binôme Khaddim : en charge de l'interface graphique ([affich\\_graph.py](#)), de l'inventaire ([inventaire.py](#)), du contrôleur ([controleur.py](#)), Images\_pieces et des effets spéciaux ([effets.py](#)).

- Travail commun : nous avons travaillé ensemble sur le fichier main.py, afin de réaliser les tests d'intégration et de vérifier la cohérence globale du programme. Un travail croisé a également été effectué sur les codes de chacun, ce qui a permis de mieux comprendre les différentes parties et d'assurer une homogénéité dans le style et la logique du projet.

## Organisation et difficultés

La rédaction de ce rapport a été discutée collectivement : nous avons choisi le plan ensemble et réalisé une synthèse de nos idées pour chaque section. La prise en main de GitHub n'a pas été facile au départ, et nos disponibilités respectives étaient parfois décalées, ce qui a compliqué la coordination. De plus, étant seulement deux, la charge de travail était conséquente. Malgré ces contraintes, nous avons réussi à mettre en place l'essentiel des fonctionnalités demandées et à produire un projet cohérent et fonctionnel.

## IX/ Conclusion

Ce projet nous a permis de mettre en pratique les concepts de la programmation orientée objet en Python à travers la réalisation d'une version simplifiée du jeu *Blue Prince*. Malgré une charge de travail conséquente et les contraintes liées à notre binôme, nous avons réussi à implémenter les fonctionnalités essentielles, à structurer le code selon une architecture claire (modèle MVC), et à produire une interface graphique fonctionnelle.

La collaboration via GitHub, bien que difficile au départ, nous a permis de progresser efficacement et de partager nos avancées. Ce travail nous a apporté une meilleure maîtrise de la POO, de la gestion de l'aléatoire et du développement collaboratif, tout en renforçant nos compétences en organisation et en synthèse.

## Remerciements

Nous tenons à remercier notre enseignant Mr Louis ANABI pour ses explications et son accompagnement tout au long du projet. Ses indications et le cahier des charges fourni nous ont permis de mieux comprendre les attentes et de structurer notre travail.

Nous remercions également nos camarades pour leurs échanges et conseils, qui nous ont aidés à progresser et à surmonter certaines difficultés techniques.

Enfin, nous souhaitons souligner l'investissement et la collaboration au sein de notre binôme. Malgré la charge de travail importante et les contraintes de disponibilité, nous avons su



travailler ensemble, partager nos idées et parvenir à une synthèse commune pour mener à bien ce projet.

